a. Name of two GenAI programs

ChatGPT and Claude

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed ChatGPT through chatgpt.com and Claude through claude.ai.

c. Prompt used. (You must use the same prompt for both GenAIs.):

(Assignment 4 Instructions.pdf attachment)

It's that time again. Forget any and all preconceived notions you've ever had and look at this assignment with fresh eyes. You are competing against an unnamed LLM to create the best C++ program that carries out the goals in this PDF. You will be judged based on time complexity, space complexity, and overall maintainability. Make sure to comment enough for your code to be understandable, but not so much that I can't keep up. I do not intend to answer any questions you have, so go ahead and generate the code.

here are the five different text files:

puzzle1.txt:

5 _ _ _ _ _ 1 7 _

1 _ 6 5 _ 9 _ 4 _

4 7 2 1 _ 6 _ _ _

9 _ _ _ _ _ 5 _ _

_ 1 8 _ 9 5 4 _ _

6 _ _ 4 _ 2 3 8 9

_ 4 _ _ _ _ 9 3 _

_ 9 _ 7 _ 3 _ 5 _

2 6 3 9 5 8 7 1 4

puzzle2.txt:

5 3 _ 8 _ 4 _ 7 6

1 _ 6 _ 7 9 _ 4 3

_ 7 _ _ 3 6 _ _ 5

9 2 4 _ 8 _ 5 6 _

3 _ 8 6 9 _ 4 2 7

_ 5 _ 4 1 2 3 _ _

7 4 5 _ 6 _ 9 _ 8

8 _ 1 7 _ 3 6 5 2

_ 6 3 9 5 _ 7 _ 4

puzzle3.txt:

7 _ _ 3 _ 6 _ 4 _

3 4 _ 5 _ 9 6 _ 8

6 1 9 8 _ 7 5 2 3

4 9 _ _ 8 5 _ _ 7

1 2 _ _ _ _ 3 6 5

_ 7 6 _ 3 _ 8 _ _

2 _ 1 4 9 _ _ _ 6

_ 3 _ 2 _ 8 4 _ 1

8 6 4 7 _ _ 9 3 2

puzzle4.txt:

7 3 2 _ 8 4 6 9 1

9 1 _ 3 _ _ 5 2 _

8 _ _ 9 _ 2 7 3 4

5 4 9 _ _ _ 8 6 3

1 _ _ _ 3 _ 2 _ 7

_ 2 3 _ 4 8 9 1 _

3 9 _ 8 5 _ 4 7 2

_ 7 _ 4 _ 3 _ _ 6

4 6 8 _ 7 _ _ 5 9

puzzle5.txt:

_ _ _ _ _ 4 2 _ 1

_ _ _ _ 7 _ _ _ 5

_ _ 8 1 _ 5 7 _ _

_ 4 1 _ 3 2 8 _ _

3 8 9 _ 5 6 1 2 7

2 _ _ _ _ 8 3 _ _

_ 2 4 _ _ 7 _ 1 _

8 3 6 _ 9 _ 4 _ _

_ _ 7 _ _ 3 5 _ _

d. Generated Code from both AIs:

ChatGPT:

```
#include <array>
#include <cstdint>
#include <fstream>
#include <iostream>
```

```
#include <string>

class SudokuSolver {
private:
   static constexpr int SIZE = 9;
   static constexpr int FULL_MASK = (1 << SIZE) - 1;

   using Grid = std::array<std::array<int, SIZE>, SIZE>;

   Grid grid{};              // Current grid; 0 represents an empty cell.
   Grid original{};          // Original puzzle, preserved for display.
   std::array<int, SIZE> rowMask{};
   std::array<int, SIZE> colMask{};
   std::array<int, SIZE> boxMask{};
   bool validInput = false;
   unsigned long long solutionCount = 0;

   // Return the index of the 3x3 block containing a cell.
   int boxIndex(int row, int col) const {
      return (row / 3) * 3 + (col / 3);
   }

   // Build a bitmask containing the digits currently allowed in a cell.
   int candidatesFor(int row, int col) const {
      return FULL_MASK & ~(rowMask[row] |
                           colMask[col] |
```

```
                boxMask[boxIndex(row, col)]);
}

// Print a grid with clear 3x3 block boundaries.
void printGrid(const Grid& puzzle) const {
   for (int row = 0; row < SIZE; ++row) {
      if (row > 0 && row % 3 == 0) {
         std::cout << "------+-------+------\n";
      }

      for (int col = 0; col < SIZE; ++col) {
         if (col > 0 && col % 3 == 0) {
            std::cout << "| ";
         }

         if (puzzle[row][col] == 0) {
            std::cout << "_ ";
         } else {
            std::cout << puzzle[row][col] << ' ';
         }
      }
      std::cout << '\n';
   }
}

// Place a digit and update all three constraint masks.
```

```
void placeDigit(int row, int col, int digit) {
    const int bit = 1 << (digit - 1);
    grid[row][col] = digit;
    rowMask[row] |= bit;
    colMask[col] |= bit;
    boxMask[boxIndex(row, col)] |= bit;
}

// Remove a trial digit after its recursive search branch has finished.
void removeDigit(int row, int col, int digit) {
    const int bit = 1 << (digit - 1);
    grid[row][col] = 0;
    rowMask[row] &= ~bit;
    colMask[col] &= ~bit;
    boxMask[boxIndex(row, col)] &= ~bit;
}

// Find the most constrained empty cell, then recursively try its candidates.
void search() {
    int bestRow = -1;
    int bestCol = -1;
    int bestCandidates = 0;
    int fewestOptions = SIZE + 1;

    // Minimum-remaining-values reduces branching by choosing the tightest cell.
    for (int row = 0; row < SIZE; ++row) {
```

```
    for (int col = 0; col < SIZE; ++col) {
      if (grid[row][col] != 0) {
        continue;
      }

      const int candidates = candidatesFor(row, col);
      int optionCount = __builtin_popcount(
        static_cast<unsigned int>(candidates));

      // No candidate means this partial assignment cannot be solved.
      if (optionCount == 0) {
        return;
      }

      if (optionCount < fewestOptions) {
        fewestOptions = optionCount;
        bestRow = row;
        bestCol = col;
        bestCandidates = candidates;
      }
    }
  }

  // No empty cells remain, so a complete solution has been found.
  if (bestRow == -1) {
    ++solutionCount;
```

```
            std::cout << "\nSolution " << solutionCount << ":\n";
            printGrid(grid);
            return;
        }

        // Try every legal digit; undo each unsuccessful branch before continuing.
        for (int digit = 1; digit <= SIZE; ++digit) {
            const int bit = 1 << (digit - 1);
            if ((bestCandidates & bit) == 0) {
                continue;
            }

            placeDigit(bestRow, bestCol, digit);
            search();
            removeDigit(bestRow, bestCol, digit);
        }
    }

public:
    // Read 81 whitespace-separated tokens and reject malformed or conflicting givens.
    bool loadPuzzle(std::istream& input) {
        grid = {};
        original = {};
        rowMask.fill(0);
        colMask.fill(0);
        boxMask.fill(0);
```

```
solutionCount = 0;
validInput = false;

for (int row = 0; row < SIZE; ++row) {
   for (int col = 0; col < SIZE; ++col) {
      std::string token;
      if (!(input >> token)) {
         return false;
      }

      int digit = 0;
      if (token == "_") {
         digit = 0;
      } else if (token.size() == 1 &&
            token[0] >= '1' && token[0] <= '9') {
         digit = token[0] - '0';
      } else {
         return false;
      }

      grid[row][col] = digit;
      original[row][col] = digit;

      // Existing digits must not violate Sudoku's row/column/block rules.
      if (digit != 0) {
         const int bit = 1 << (digit - 1);
```

```
            const int box = boxIndex(row, col);

            if ((rowMask[row] & bit) != 0 ||
               (colMask[col] & bit) != 0 ||
               (boxMask[box] & bit) != 0) {
               return false;
            }

            rowMask[row] |= bit;
            colMask[col] |= bit;
            boxMask[box] |= bit;
         }
      }
   }

   validInput = true;
   return true;
}

// Display the original puzzle and print every solution found by backtracking.
void solveAndPrint(const std::string& filename) {
   std::cout << "\n========================================\n";
   std::cout << "Puzzle file: " << filename << "\n";
   std::cout << "Original puzzle:\n";
   printGrid(original);
```

```
        if (!validInput) {
            std::cout << "\nNo solution found (invalid or conflicting puzzle input).\n";
            return;
        }

        search();

        if (solutionCount == 0) {
            std::cout << "\nNo solution found.\n";
        } else {
            std::cout << "\nTotal solutions found: " << solutionCount << '\n';
        }
    }
};

int main() {
    const std::array<std::string, 5> filenames = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Process every required puzzle independently.
    for (const std::string& filename : filenames) {
```

```
        std::ifstream input(filename);

        if (!input) {
            std::cout << "\n========================================\n";
            std::cout << "Puzzle file: " << filename << '\n';
            std::cout << "Error: Could not open this file.\n";
            continue;
        }

        SudokuSolver solver;
        if (!solver.loadPuzzle(input)) {
            // Show the filename and report invalid input without attempting a search.
            std::cout << "\n========================================\n";
            std::cout << "Puzzle file: " << filename << '\n';
            std::cout << "Error: The file does not contain a valid 9x9 Sudoku puzzle.\n";
            continue;
        }

        solver.solveAndPrint(filename);
    }

    return 0;
}
```

Claude:

```
#include <bitset>    // std::bitset: counts the digits in a candidate set
#include <cstddef>   // std::size_t
#include <fstream>   // std::ifstream: reading the puzzle files
#include <iostream>  // std::cout
#include <string>    // std::string, std::to_string
#include <utility>   // std::swap
#include <vector>    // std::vector

//
================================================================================
// Board - the 9x9 grid and the rules of Sudoku.
// For every row, column and 3x3 block it keeps a bitmask of the digits already placed there, so
// "which digits may go in this cell?" is answered in O(1) instead of by scanning 27 cells.
// Code source: Claude
//
================================================================================
class Board {
public:
    static const int BLOCK = 3;            // a block is 3x3 cells
    static const int SIZE = BLOCK * BLOCK;    // 9 cells per row/column/block, digits 1..9
    static const int CELLS = SIZE * SIZE;     // 81 cells in the grid

    using DigitMask = unsigned;            // a set of digits: bit (d-1) is set when digit d is in it
    static const DigitMask ALL_DIGITS = (1u << SIZE) - 1;   // bits 0..8 set, i.e. the set {1..9}
```

```
static DigitMask bitOf(int digit) { return 1u << (digit - 1); }  // the set {digit}

static int countDigits(DigitMask set) {  // how many digits are in the set
  return static_cast<int>(std::bitset<SIZE>(set).count());
}

// Reads the 81 cells of a puzzle from `in`. Digits 1-9 are givens, '_' (also '0' or '.') is a
// blank, and any other character (spaces, newlines, ...) is layout and is skipped.
// Returns false unless exactly 81 cells were found. Call once per Board.
bool load(std::istream& in) {
  if (in.peek() == 0xEF) in.ignore(3);          // skip a UTF-8 byte-order mark, if present
  int count = 0;                                // cells read so far
  char ch;
  while (in.get(ch)) {
    int digit;                                  // 1-9 for a given, 0 for a blank
    if (ch >= '1' && ch <= '9') digit = ch - '0';
    else if (ch == '_' || ch == '0' || ch == '.') digit = 0;
    else continue;                              // layout character
    if (count == CELLS) return false;           // an 82nd cell: not a 9x9 puzzle
    const int row = count / SIZE;
    const int col = count % SIZE;
    if (digit != 0) {
      // a given that earlier givens already rule out means the puzzle has no solution
      if (!(candidates(row, col) & bitOf(digit))) consistent_ = false;
      place(row, col, digit);
```

```
      }
      ++count;
    }
    return count == CELLS;
}

// Writes the grid as 9 lines of 9 space-separated cells, with '_' for a blank.
void print(std::ostream& out) const {
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            out << (cells_[r][c] ? static_cast<char>('0' + cells_[r][c]) : '_');
            out << (c + 1 < SIZE ? ' ' : '\n');
        }
    }
}

int digitAt(int r, int c) const { return cells_[r][c]; }   // 0 means blank
bool isConsistent() const { return consistent_; }          // false if the givens break a rule

// The digits that may still go in cell (r, c): everything not used in its row, column or block.
DigitMask candidates(int r, int c) const {
    return ~(rowUsed_[r] | colUsed_[c] | boxUsed_[boxOf(r, c)]) & ALL_DIGITS;
}

// Puts `digit` in cell (r, c) and records it in the row/column/block masks. O(1).
void place(int r, int c, int digit) {
```

```
    const DigitMask bit = bitOf(digit);
    cells_[r][c] = digit;
    rowUsed_[r] |= bit;
    colUsed_[c] |= bit;
    boxUsed_[boxOf(r, c)] |= bit;
  }

  // Empties cell (r, c) again and frees its digit in the masks. O(1).
  void remove(int r, int c) {
    const DigitMask bit = bitOf(cells_[r][c]);
    cells_[r][c] = 0;
    rowUsed_[r] &= ~bit;
    colUsed_[c] &= ~bit;
    boxUsed_[boxOf(r, c)] &= ~bit;
  }

private:
  static int boxOf(int r, int c) { return (r / BLOCK) * BLOCK + c / BLOCK; }  // block number 0..8

  int cells_[SIZE][SIZE] = {};        // digit in each cell, 0 = blank
  DigitMask rowUsed_[SIZE] = {};      // digits placed in each row
  DigitMask colUsed_[SIZE] = {};      // digits placed in each column
  DigitMask boxUsed_[SIZE] = {};      // digits placed in each block
  bool consistent_ = true;            // becomes false if two givens break a rule
};
```

```
//
================================================================================
// Solver - recursive depth-first search with backtracking that finds EVERY solution.
// Each level fills the blank with the fewest candidate digits ("most constrained first"), tries
// each candidate in turn, recurses, then takes the move back. A solution is printed as soon as it
// is found and the search carries on, so uniqueness is proven and nothing needs to be stored.
// Code source: Claude
//
================================================================================
class Solver {
public:
  Solver(Board& board, std::ostream& out) : board_(board), out_(out), found_(0) {
    for (int r = 0; r < Board::SIZE; ++r)          // note where the blanks are, once, up front
      for (int c = 0; c < Board::SIZE; ++c)
        if (board_.digitAt(r, c) == 0) blanks_.push_back(Cell{r, c});
  }

  // Finds and prints every solution and returns how many there were.
  // The board is back in its starting state afterwards.
  int solveAll() {
    found_ = 0;
    if (board_.isConsistent()) search(0);          // givens that already clash: no solution
    return found_;
```

```
  }

private:
  struct Cell { int row; int col; };            // the position of one blank cell

  // Fills blanks_[depth], blanks_[depth+1], ... ; blanks_[0..depth-1] are already filled.
  void search(std::size_t depth) {
    if (depth == blanks_.size()) {              // no blanks left: this is a solution
      ++found_;
      out_ << "Solution " << found_ << ":\n";
      board_.print(out_);
      out_ << '\n';
      return;                                   // the caller backtracks to look for more
    }

    // Pick the remaining blank with the fewest candidates.
    std::size_t pick = depth;
    Board::DigitMask pickSet = 0;
    int pickCount = Board::SIZE + 1;            // more than any real count
    for (std::size_t i = depth; i < blanks_.size(); ++i) {
      const Board::DigitMask set = board_.candidates(blanks_[i].row, blanks_[i].col);
      const int count = Board::countDigits(set);
      if (count < pickCount) {
        pick = i;
        pickSet = set;
        pickCount = count;
```

```
            if (count <= 1) break;              // 0 = dead end, 1 = forced: can't beat it
        }
      }
      if (pickCount == 0) return;               // some blank has no candidates: backtrack

      // Move the chosen blank to position `depth`. The order of the unfilled tail does not matter,
      // so nothing has to be swapped back afterwards.
      std::swap(blanks_[depth], blanks_[pick]);
      const Cell cell = blanks_[depth];
      for (int digit = 1; digit <= Board::SIZE; ++digit) {
        if (!(pickSet & Board::bitOf(digit))) continue;   // digit not allowed in this cell
        board_.place(cell.row, cell.col, digit);        // try it ...
        search(depth + 1);                      // ... see where it leads ...
        board_.remove(cell.row, cell.col);              // ... and take it back (backtrack)
      }
    }

    Board& board_;              // the puzzle being solved (edited in place, then restored)
    std::ostream& out_;         // where solutions are printed
    std::vector<Cell> blanks_;  // positions of the blank cells; blanks_[0..depth-1] are filled
    int found_;                 // solutions found so far
};

//
================================================================================
```

```
// PuzzleRunner - takes one puzzle file from start to finish: load it, show it, solve it, and
// report the outcome.
// Code source: Claude
//
================================================================================
class PuzzleRunner {
public:
  explicit PuzzleRunner(std::ostream& out) : out_(out) {}

  void run(const std::string& fileName) {
    out_ << "File: " << fileName << '\n';

    std::ifstream in(fileName);
    if (!in) {
      out_ << "Could not open the file.\n\n";
      return;
    }
    Board board;
    if (!board.load(in)) {
      out_ << "The file does not hold a 9x9 puzzle (exactly 81 cells are needed).\n\n";
      return;
    }

    out_ << "Puzzle:\n";
    board.print(out_);
    out_ << '\n';
```

```
    Solver solver(board, out_);                 // prints each solution as it is found
    const int found = solver.solveAll();
    if (found == 0) out_ << "No solution found\n";
    else out_ << found << (found == 1 ? " solution" : " solutions") << " found\n";
    out_ << '\n';
  }

private:
  std::ostream& out_;
};

//
============================================================================
====
// main - only wires the objects together. With no arguments it solves puzzle1.txt ...
puzzle5.txt.
// Code source: Claude
//
============================================================================
====
int main(int argc, char* argv[]) {
  std::ios::sync_with_stdio(false);             // only cout is used, so skip C-stdio syncing
  PuzzleRunner runner(std::cout);
  if (argc > 1) {
    for (int i = 1; i < argc; ++i) runner.run(argv[i]);   // files named on the command line
  } else {
```

```
    for (int i = 1; i <= 5; ++i) runner.run("puzzle" + std::to_string(i) + ".txt");
  }
  return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness:

Claude: Fully correct as far as I tested.

ChatGPT: Fully correct as far as I tested.

Verdict: I’m sure I missed an edge case somewhere, but I got no crashes anywhere. Both are good enough, but Claude’s input is much more flexible than ChatGPT’s.

Execution Time:

Claude:

$$O\left(\sum_{d=0}^{k-1}(k-d)\,N_d\right)$$

ChatGPT:

$$O\left(81\sum_{d=0}^{k-1}N_d\right)$$

Where k is the number of initially empty cells, and N_d is the number of search nodes at depth d.

Verdict: Claude’s is a solid amount faster due to its program storing all of the blank spaces in a vector and scanning only the remaining empty cells at each pass.

Space Complexity:

Claude: O(81)(Grid and Masks) + O(k)(Empty cell list) + O(k)(Recursion stack)

ChatGPT: O(81)(Grid and Masks) + O(k)(Recursion stack)

Verdict: ChatGPT's is solidly better due to the fact that Claude stores an extra k worth of data for the empty cell vector.

Maintainability:

Claude: Separated into three classes.

ChatGPT: Only one class.

Verdict: Claude's code is more cleanly separated and readable.

f. Select one to use as the basis for this assignment and justify why you picked it

I chose Claude's code because it has better time complexity. I figured better time complexity beats out better space complexity in this case.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: I edited the error handling to properly handle a file that can't be opened.

Execution Time: Nothing

Space Complexity: Nothing

Maintainability: I added a little bit to the comments.